

AV1 — IA para Cibersegurança · Relatório

Aluno: Rafael Holder (individual) · **Notebook:** `aula5_modelagem_detector_malware_uci_splunk.ipynb` · CESAR School, 2026.2

Contexto

O notebook treina um Random Forest para separar executáveis Windows benignos de malware usando a base UCI 541 (6.248 amostras, 90,5% malware, 28 features em três famílias: estrutura, entropia/seções e imports). Ao auditar a base, verifiquei que as 18 colunas de entropia, imports e `number_of_sections` têm apenas 2 valores diferentes de zero em cerca de 112 mil células. Como todo PE tem pelo menos uma seção, esses zeros são dado ausente, e o modelo acaba decidindo só com features de tamanho e layout do arquivo. Além disso, benignos e malwares vieram de fontes diferentes (benignos de *Program Files* do Windows 7/8; malwares de VxHeaven e VirusTotal 2018), então a origem quase determina o rótulo.

Alteração 1 — Dados e Features

O QUE fiz. Retreinei o modelo em duas variantes: **1a**, sem as 18 features zeradas (sobram 10), e **1b**, sem as zeradas e também sem as 5 features de layout que podem denunciar a fonte da coleta (`image_base`, `file_alignment`, `section_alignment`, `size_of_headers`, `size_of_optional_header`), sobrando 5. A 1a ficou igual ao original: recall de benigno 0,832 nas duas, PR-AUC 0,997 nas duas e FN 48 contra 42. Na 1b o recall de benigno caiu de 0,832 para 0,588, os falsos positivos foram de 20 para 49 e o ROC-AUC caiu de 0,973 para 0,944. O recall de malware ficou estável (0,959).

COMO fiz. Criei a seção "AV1 — Experimentos" no fim do notebook, sem editar nenhuma célula original. A função `escolher_limiar` repete a regra da seção 9 (recall $\geq$ 95% com maior precisão), e `resumo` calcula FN, FP, recall por classe, PR-AUC e ROC-AUC no teste. Nas duas variantes usei o mesmo split 60/20/20, o mesmo `RANDOM_SEED = 42` e os mesmos hiperparâmetros (250 árvores, `min_samples_leaf=2`, `class_weight='balanced'`). Cada variante escolhe o próprio limiar só na validação (0,40 na 1a e 0,41 na 1b), e o teste apenas reporta.

POR QUE fiz. A pergunta era se o modelo detecta malware ou detecta de onde o arquivo veio. A 1a mostra que as features zeradas eram só ruído. A 1b mostra que, sem as marcas de layout da coleta, o modelo perde a capacidade de reconhecer os benignos. Ou seja, boa parte do desempenho vinha de reconhecer a "cara" dos programas instalados, não de comportamento malicioso. Isso também está ligado à evasão: tamanhos, alinhamentos e endereços são baratos de alterar com padding ou recompilação.

Alteração 2 — Avaliação (split por procedência, sem vazamento de fonte)

O QUE fiz. Troquei o split aleatório por um split **por procedência do malware**: treino o mesmo Random Forest com o malware de uma fonte e testo no malware da outra (o benigno, que vem de uma única fonte, dividido ao meio). Faço as duas direções e comparo com o split aleatório original (PR-AUC 0,997, ROC-AUC 0,973, recall de malware 0,963). Treino em VxHeaven / teste em VirusTotal: PR-AUC 0,982, ROC-AUC 0,871 e recall de malware a 0,50 de apenas **0,606** (deixa passar ~39% do malware). Treino em VirusTotal / teste em VxHeaven: PR-AUC 0,984, ROC-AUC 0,901 e recall de malware 0,867.

COMO fiz. Recuperei a coluna `source_dataset` de cada amostra (o `df` preserva o índice que sobrevive ao split) e montei os conjuntos por fonte, mantendo o mesmo seed e os mesmos hiperparâmetros do modelo original. Como aqui não há uma validação separada para calibrar limiar, comparo o **poder de discriminação por PR-AUC e ROC-AUC**, que independem do limiar, e mostro o recall a 0,50 apenas como ilustração. Nenhuma decisão olha o teste para ajustar o modelo.

POR QUE fiz. O split aleatório deixa variantes próximas da mesma coleção caírem em treino e teste, o que infla a métrica (vazamento) e esconde o viés de origem. Separar por procedência responde à pergunta que importa para cibersegurança: o detector generaliza para malware de outra fonte, ou só

reconhece a assinatura da coleção onde treinou? É o teste de estresse direto do achado da Alteração 1 e segue o protocolo da disciplina de evitar vazamento no split. O PR-AUC cai pouco (0,997 → 0,982/0,984) porque cada teste ainda é ~90% malware, o que mantém o piso alto; mas o ROC-AUC cai de 0,973 para 0,871/0,901 e o recall de malware a 0,50 desaba para 0,606/0,867. A queda é clara: boa parte do desempenho do split aleatório vinha de reconhecer a procedência, e o modelo não generaliza bem para malware de outra coleção.

Tabela antes × depois (conjunto de teste)

Experimento	nº feat.	limiar	recall malware	recall benigno	FN	FP	PR-AUC	ROC-AUC
Original (28 features) — split aleatório	28	0,37	0,963	0,832	42	20	0,997	0,973
Alt. 1a: sem zeradas	10	0,40	0,958	0,832	48	20	0,997	0,974
Alt. 1b: sem zeradas e sem origem	5	0,41	0,959	0,588	46	49	0,994	0,944
Alt. 2: treino VxHeaven / teste VirusTotal	28	0,50	0,606	0,943	1163	17	0,982	0,871
Alt. 2: treino VirusTotal / teste VxHeaven	28	0,50	0,867	0,799	360	60	0,984	0,901

Limitações

- **Viés de origem:** a fonte da coleta e o rótulo são praticamente a mesma coisa. A Alteração 2 mede o quanto isso contamina o desempenho, mas não o elimina: o benigno continua vindo de uma única fonte.
- **Vazamento residual:** mesmo separando por fonte de malware, variantes da mesma família dentro de uma fonte ainda podem se repetir. Dividir por família/hash seria mais rigoroso.
- **Idade da base:** malwares antigos (VxHeaven) e de 2018, com benignos do Windows 7/8. O resultado não vale para ameaças atuais.
- **Poucos benignos:** são ~595 no total, então cada FP pesa bastante no recall de benigno; diferenças pequenas (ex.: 42 contra 48 FN) estão dentro do ruído.
- **Reprodutibilidade:** executei localmente com scikit-learn 1.9.1; no Colab o limiar e os números variam um pouco (o limiar da aula foi 0,46 no Colab contra 0,37 local). As conclusões são as mesmas. A Alteração 2 foi calculada no mesmo scikit-learn 1.9.1, e o baseline reproduz exatamente os números do notebook (limiar 0,37, PR-AUC 0,997, ROC-AUC 0,973).